

Light

1 Importance Sampling of Many Lights on the GPU

1.1 整体思路

该方法出自 Ray Tracing Gems 的第 18 章，主要描述了如何在 GPU 上对大量光源进行重要性采样。该方法的核心思想是使用层次化的包围盒 (Bounding Volume Hierarchy, BVH) 来组织光源，构造树状结构，根据采样点 (着色点) 遍历树，并根据重要性选择子节点进行递归采样，直到到达叶节点 (光源)。

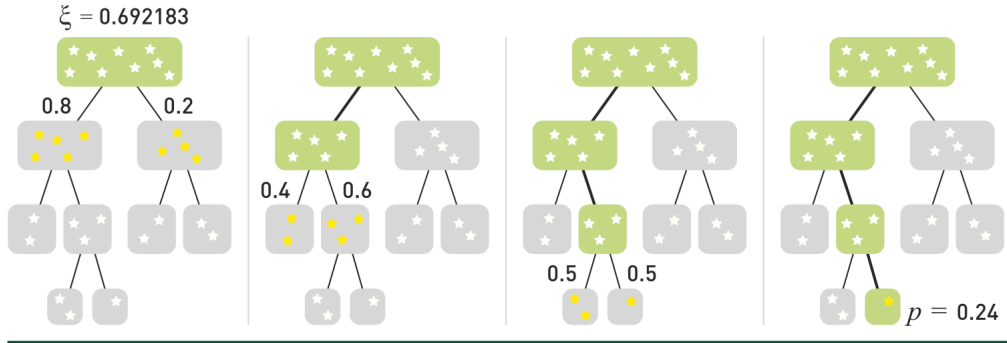


图 1: Light BVH Sample

1.2 算法流程

1.2.1 光源预处理

将每个光源看作 mesh，处理的基本单位是单个三角形，我们为每个三角形 i 预计算辐射量 flux Φ_i ：

$$\Phi_i = \iint_{\Omega} L_i(X)(\mathbf{n}_i \cdot \omega) d\omega dA$$

其中 $L_i(X)$ 是光源 i 在点 X 处的辐射亮度， $\mathbf{n}_i$ 是光源 i 的法线， ω 是从光源指向采样点的方向。

如果光源强度在三角形表面为常数，对应的通量为 $\Phi_i = \pi A_i L_i$ ，其中 A_i 是三角形 i 的面积。如果该三角形带有纹理贴图，则需要对该三角形进行预计算 flux，存储在纹理中，应用三角形的 uv 坐标采样 flux。

1.2.2 构建 BVH

(1) Light Orientation Cone

在二叉树的每个子节点存储一个 Light Orientation Cone，由一条方向轴和两个角度 θ_o 和 θ_e 组成，前者描述当前节点内所有发光体的法线，后者描述所有发光体的发光方向。

比如，对于单面发光的三角形，对应的 $\theta_o = 0$ （只有一个法线）， $\theta_e = \frac{\pi}{2}$ （在半球方向内进行发光）。对于发光的球，对应的 $\theta_o = \pi$ （法线指向所有方向）， $\theta_e = \frac{\pi}{2}$ （在每个法线处朝半球方向发光）。

(2) 定义 split plane

(2.1) SAH(Surface Area Heuristic)

将左子节点的集合记作 $L = \cup_{j=0}^i bin_j$ ，右子节点的集合记作 $R = \cup_{j=i+1}^k bin_j$ 。则该划分的代价为：

$$cost(L, R) = \frac{n(L)a(L) + n(R)a(R)}{n(L \cup R)a(L \cup R)}$$

其中 $n(L)$ 和 $n(R)$ 分别是左、右子节点中的光源数量， $a(L)$ 和 $a(R)$ 分别是左、右子节点的包围盒面积。

(2.2) SAOH(Surface Area Orientation Heuristic)

SAOH 在 SAH 的基础上引入了光源的方向信息，定义如下：

$$cost(L, R, s) = k_r(s) \frac{\Phi(L)a(L)M_\Omega(L) + \Phi(R)a(R)M_\Omega(R)}{a(L \cup R)M_\Omega(L \cup R)}$$

其中 s 是 split plane 所在的轴, $k_r(s) = \frac{length_{max}}{length_s}$ 用于处理过薄的 box; M_Ω 表示光源朝向的相似程度。

(2.3) VH (Volume Heuristic)

VH 在 SAH 的基础上用 AABB 的体积代替面积。

(2.4) VOH (Volume Orientation Heuristic)

VOH 在 SAOH 的基础上用 AABB 的体积代替面积。

1.2.3 光源重要性采样

构建完 Light BVH 后, 在进行光照计算时, 需要进行光源采样, 我们用随机数对 Light BVH 进行采样, 选择具体的单个光源进行光照计算。

考虑以下因素:

(1) Distance: 着色点和节点对应的 AABB 的中心的距离, 距离越近, 重要性越高。但是在 BVH 的较高 level 时, 此时 AABB 可能包含场景的大部分空间, 对重要性的估计准确度较低。

(2) Light Flux: 节点的 flux 为节点内所有光源发出的 flux 之和。

(3) Light Orientation: 节点的 Light Orientation Cone 对应的主方向, 跟节点 AABB 的中心和着色点构成的方向的夹角, 夹角越小, 重要性越高。

(4) Light Visibility: 光照方向和着色点法线之间的夹角, 如果大于 90 度, 则属于无效光照计算。还需要考虑场景物体对光线的遮挡。

给定着色点 X 和子节点 C , 我们可以计算节点的重要性:

$$importance(X, C) = \frac{\Phi(C)|\cos\theta'_i|}{\|X - C\|^2} \times \begin{cases} \cos\theta', & \text{if } \theta' < \theta_e \\ 0, & \text{otherwise} \end{cases}$$

如图2所示, 从着色点 X 出发, 指向节点 AABB 的中心, 以此作为圆锥的轴, 构造能够包含节点 AABB 的最小圆锥。其中 $\theta'_i = \max(0, \theta_i - \theta_u)$, $\theta' = \max(0, \theta - \theta_o - \theta_u)$ 。

分别计算左子节点和右子节点的重要性, 选择左子节点的概率为:

$$p(L) = \frac{importance(L)}{importance(L) + importance(R)}$$

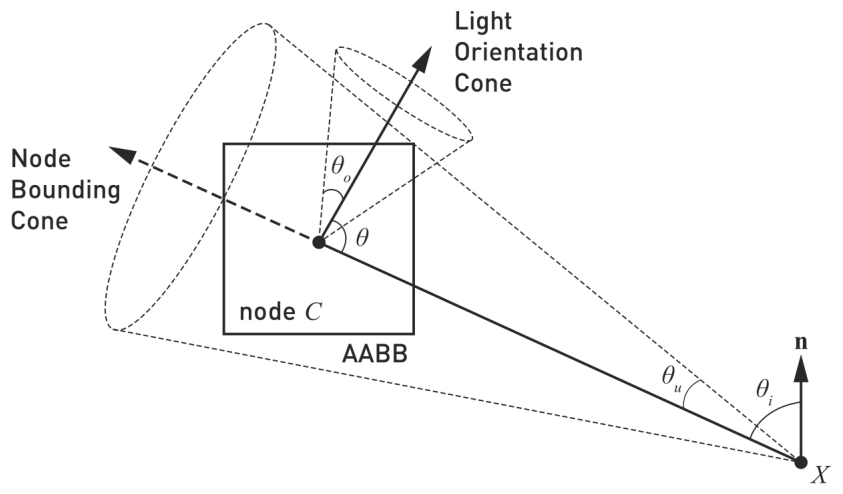


图 2: Node Importance

对 Light BVH 进行采样，会同时传入随机数 ξ ，如果 $\xi < p(L)$ ，则选择左子节点，否则选择右子节点。

除此之外，每次选择新的子节点后，会更新随机数，更新方法如下：

$$\xi' = \begin{cases} \frac{\xi}{p(L)}, & \text{if } \xi < p(L) \\ \frac{\xi - p(L)}{p(R)}, & \text{otherwise} \end{cases}$$

1.3 个人应用记录

本人在自研离线渲染引擎的开发工作中应用过该多光源采样算法，提高了渲染效率，渲染效果也能满足业务需求。此处记录对该方法的修改和思考：

(1) 预处理改为对每个光源计算 AABB，而不是用每个三角形，一方面是考虑到 IES 灯和 Spot Light，用三角面片不合适；另一方面，如果光源带有纹理贴图，预处理比较耗时；主要还是最终效果已经满足需求。如果是一般的 mesh light（非 area light 和 sphere light），则使用三角面片进行处理。

(2) 帧间信息复用，如果渲染动画或者视频，在场景光源参数不变的情况下，Light BVH 可以复用；但是，如果灯光本身带有动画，需要重新构建

Light BVH。

(3) 每次进行光照计算的时候，只对 Light BVH 采样一次，这属于比较激进的策略。

1.4 参考

Ray Tracing Gems

2 Real-Time Stochastic Lightcuts

3 基本思想

本文是 stochastic lightcuts 在 GPU 上的实现，其主要优化点有两个：

(1) 构建完美二叉树，应用于光源采样，不仅构建速度快，而且遍历效率高；

(2) 在 $k \times k$ 的像素块内共享 light cuts 的结果，减少 light cuts 带来的计算量。

3.1 构建完美平衡二叉树

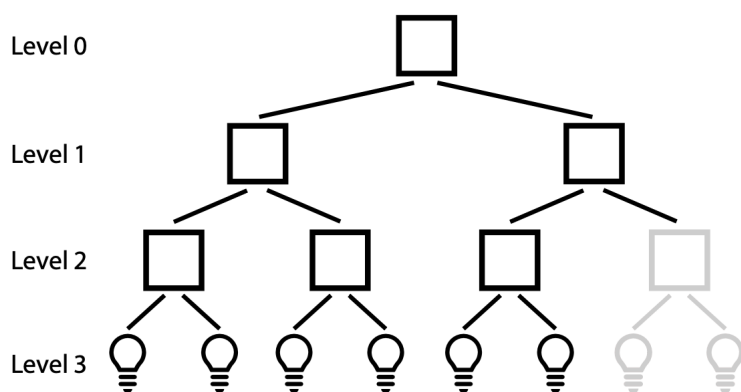


图 3: Perfected Balanced Light Tree

如图3所示，用完美平衡二叉树构建 Light Tree，我们不必储存父节点和子节点的映射关系（指针），根据储存索引的关系，根据父节点的索引得

到子节点的索引。

(1) 根据所有的光源的位置，生成对应的 Morton Code，根据 Morton Code 进行排序，将光源填充到 Light Tree 的叶子节点中（当光源数量不是 2 的幂时，会进行填充）。

(2) 从叶子节点开始，向上遍历，每个节点存储 intensity 和 AABB，父节点的 AABB 为子节点的 AABB 的并集，intensity 为子节点的 intensity 之和。

3.2 遍历 Light Tree

选择子节点的策略为根据着色点，对每个子节点计算两个权重：

$$w_j^{min} = \frac{F_j(\mathbf{x}, \omega) \|\mathbf{I}_j\|}{(d_j^{min}(\mathbf{x}))^2}$$

$$w_j^{max} = \frac{F_j(\mathbf{x}, \omega) \|\mathbf{I}_j\|}{(d_j^{max}(\mathbf{x}))^2}$$

其中 $\mathbf{x}$ 是着色点， $F_j(\mathbf{x}, \omega)$ 为 reflectance bound， $\mathbf{I}_j$ 为节点的 intensity， $d_j^{min}(\mathbf{x})$ 和 $d_j^{max}(\mathbf{x})$ 为着色点到节点 AABB 的最小距离和最大距离。

给定子节点 j 和 k ，为选择子节点 j 计算两个概率：

$$p_j^{min} = \frac{w_j^{min}}{w_j^{min} + w_k^{min}}$$

$$p_j^{max} = \frac{w_j^{max}}{w_j^{max} + w_k^{max}}$$

最终选择子节点 j 的概率为 $p_j = \frac{p_j^{min} + p_j^{max}}{2}$ 。

注意点：

(1) 如果着色点在节点 AABB 内， d_j^{min} 和 d_j^{max} 均为 0，那么在计算权重时忽略距离项；

(2) 计算最小距离和最大距离，是为了避免着色点距离 AABB 过近或过远的情况，导致子节点选择权重过小。（一种 trick 的技巧）

3.3 共享采样结果

利用空间上的相似性，相邻像素共享 light cut 的结果，可以减少计算量，其流程如下：

(1) 对 $k \times k$ 的像素块, 选择最具代表性的像素 (几何属性和材质属性), 执行 light cut;

(2) $k \times k$ 的像素块内的每个像素使用相同的 light cut 结果。

3.4 个人应用记录

本人在自研引擎的工作中应用过该算法, 相比于 Light BVH, 该算法在灯光采样的时间上能节省约 15% 的时间, 效果上和 Light BVH 各有优劣。

3.5 参考

Real-Time Stochastic Lightcuts

Stochastic Lightcuts

4 ReSTIR

4.1 基本思想

4.2 前提技术

4.2.1 Resampled Importance Sampling

当我们要计算如下积分:

$$I = \int_{\Omega} f(x) d\mu(x)$$

我们有两个概率密度函数 (Probability Density Function, PDF), source PDF 为 p , 方便采样, 但是对 f 的估计效果较差; sampling PDF 为 g , 对 f 的估计效果较好, 但是采样困难。

标准的重要性采样只使用 p , 逼近效果较差, 这时可以考虑将重要性采样延伸到重要性重采样 (Resampled Importance Sampling, RIS)。它的操作流程如下:

- (1) 从 pdf p 生成 $M (M \geq 1)$ 个样本, 得到样本集合 $\mathbf{X} = (X_1, \dots, X_M)$;
- (2) 为每个样本计算权重 $\omega(X_j) = \frac{g(X_j)}{p(X_j)}$;
- (3) 根据权重从样本集合里 $\mathbf{X}$ 生成 N 个样本 $(Y_1, \dots, Y_N)$;
- (4) 按下式计算积分:

$$I_{RIS} = \frac{1}{N} \sum_{i=1}^N \left(\frac{f(Y_i)}{g(Y_i)} \cdot \frac{1}{M} \sum_{j=1}^M \omega(X_j) \right)$$

注：要保证 RIS 是无偏的，需要保持 g 和 p 处处大于等于 0。
原论文中的伪代码如图4所示。

Algorithm 1: Resampled importance sampling.

Input : M, q : number of candidates to generate ($M \geq 1$) for pixel q .
Output: Sample y and the sum of RIS weights $\sum_{i=1}^M w(x_i)$

```

1 // Generate proposals  $\mathbf{x} = \{x_1, \dots, x_M\}$ 
2  $\mathbf{x} \leftarrow \emptyset$ 
3  $\mathbf{w} \leftarrow \emptyset$ 
4  $w_{\text{sum}} \leftarrow 0$ 
5 for  $i \leftarrow 1$  to  $M$  do
6   generate  $x_i \sim p$ 
7    $\mathbf{x} \leftarrow \mathbf{x} \cup \{x_i\}$ 
8    $w_i \leftarrow \hat{p}_q(x_i)/p(x_i)$ 
9    $w_{\text{sum}} \leftarrow w_{\text{sum}} + w_i$ 
10   $\mathbf{w} \leftarrow \mathbf{w} \cup \{w_i\}$ 
11 // Select from candidates  $\mathbf{x}$ 
12 Compute normalized CDF  $C$  from  $\mathbf{w}$ 
13 draw random index  $z \in [0, M)$  using  $C$  to sample  $\propto w_z$ 
14  $y \leftarrow x_z$ 
15 return  $y, w_{\text{sum}}$ 

```

图 4: RIS

4.2.2 Weighted Reservoir Sampling

加权蓄水池采样 (Weighted Reservoir Sampling, WRS) 是用于在数据流中采样 N 个随机样本的算法。蓄水池中固定保持 N 个样本，对数据流中的每个元素只处理一次，考虑是否保留，替换蓄水池中的样本，无需考虑数据流的数据总量。

当已经处理了 m 个样本后，样本 x_i 出现在蓄水池中的概率为：

$$\frac{\omega(x_i)}{\sum_{j=1}^m \omega(x_j)}$$

对于第 $m+1$ 个样本，替换 x_i 的概率为：

$$\frac{\omega(x_{m+1})}{\sum_{j=1}^{m+1} \omega(x_j)}$$

那么之前的样本 x_i 仍在蓄水池中的概率为：

$$\frac{\omega(x_i)}{\sum_{j=1}^m \omega(x_j)} \left(\frac{\omega(x_{m+1})}{\sum_{j=1}^{m+1} \omega(x_j)} \right) = \frac{\omega(x_i)}{\sum_{j=1}^{m+1} \omega(x_j)}$$

如此保持蓄水池每个样本被替换的概率与其权重呈正相关。原论文中的伪代码如图5所示。

Algorithm 2: Weighted reservoir sampling.

```

1 class Reservoir
2    $y \leftarrow 0$  // The output sample
3    $w_{\text{sum}} \leftarrow 0$  // The sum of weights
4    $M \leftarrow 0$  // The number of samples seen so far
5   function update( $x_i, w_i$ )
6      $w_{\text{sum}} \leftarrow w_{\text{sum}} + w_i$ 
7      $M \leftarrow M + 1$ 
8     if rand() < ( $w_i / w_{\text{sum}}$ ) then
9        $y \leftarrow x_i$ 
10 function reservoirSampling( $\mathbb{S}$ )
11   Reservoir  $r$ 
12   for  $i \leftarrow 1$  to  $M$  do
13      $r.\text{update}(\mathbb{S}[i], \text{weight}(\mathbb{S}[i]))$ 
14   return  $r$ 

```

图 5: WRS

4.3 Streaming RIS With Spatiotemporal Reuse

4.3.1 Streaming RIS using reservoir sampling

将 WRS 应用于 RIS，将 RIS 变为流式算法。如图6所示，每个 pixel q 包含一个蓄水池，给它输入 M 次由 RIS 生成的光源样本后，蓄水池中的 y 就是最终的光源样本（只有该样本需要进行 shadow ray 测试），然后以下式计算对 pixel 的贡献：

$$\frac{f(y)}{\hat{p}(y)} \left(\frac{1}{M} \sum_{j=1}^M \omega(x_j) \right)$$

Algorithm 3: Streaming RIS using weighted reservoir sampling.

```

1 foreach pixel  $q \in \text{Image}$  do
2   | Image[ $q$ ]  $\leftarrow$  shadePixel(RIS( $q$ ),  $q$ )
3 function RIS( $q$ )
4   | Reservoir  $r$ 
5   | for  $i \leftarrow 1$  to  $M$  do
6   |   | generate  $x_i \sim p$ 
7   |   |  $r.\text{update}(x_i, \hat{p}_q(x_i)/p(x_i))$ 
8   |   |  $r.W = \frac{1}{\hat{p}_q(r.y)} \left( \frac{1}{r.M} r.w_{\text{sum}} \right)$  // Equation (6)
9   |   | return  $r$ 
10 function shadePixel(Reservoir  $r$ ,  $q$ )
11 | return  $f_q(r.y) \cdot r.W$ 

```

图 6: Streaming RIS using reservoir sampling

4.3.2 时空复用

(1) 空间复用：相邻像素对应的几何特性和材质特性具有一定相似性，可以根据这个特点进行样本复用，步骤如下：

(1.1) 对于每个像素 q ，应用 RIS 生成 M 个样本，然后应用 WRS 将结果进行存储；

(1.2) 每个像素选择 k 个相邻像素的采样结果，将多个蓄水池的结果进行结合，如图7所示。

Algorithm 4: Combining the streams of multiple reservoirs.

Input :Reservoirs r_i to combine.
Output:A combined reservoir equivalent to the concatenated input streams of $r_1, \dots, r_k$.

```

1 function combineReservoirs( $q, r_1, r_2, \dots, r_k$ )
2   | Reservoir  $s$ 
3   | foreach  $r \in \{r_1, \dots, r_k\}$  do
4   |   |  $s.\text{update}(r.y, \hat{p}_q(r.y) \cdot r.W \cdot r.M)$ 
5   |   |  $s.M \leftarrow r_1.M + r_2.M + \dots + r_k.M$ 
6   |   |  $s.W = \frac{1}{\hat{p}_q(s.y)} \left( \frac{1}{s.M} s.w_{\text{sum}} \right)$  // Equation (6)
7   |   | return  $s$ 

```

图 7: Combine Multiple Reservoirs

每个像素用 $O(k + M)$ 的时间对 $k \cdot M$ 个候选样本进行有效查找；可以进行迭代， n 次空间复用，可以用 $O(nk + M)$ 的时间对 $k^n M$ 个样本进行查找。

(2) 时间复用：存储历史帧的采样结果，将当前像素投影到上一帧的位置，应用历史位置周围的采样结果。

(3) 可见性复用：当累计的样本数量达到一定程度时，估计的分布逼近的是无遮挡的路径贡献，这时 visibility 是 noise 生成的主要原因。为了处理该问题，对每个像素的蓄水池产生的样本 y 进行可见性测试。如果 y 被遮挡了，抛弃该蓄水池。

Algorithm 5: Our algorithm for RIS with spatiotemporal reuse.

Input : Image sized buffer containing the previous frame's reservoirs

Output : The current frame's reservoirs

```

1 function reservoirReuse(prevFrameReservoirs)
2   reservoirs  $\leftarrow$  new Array[ImageSize]
3   // Generate initial candidates
4   foreach pixel  $q \in$  Image do
5     | reservoirs[ $q$ ]  $\leftarrow$  RIS( $q$ ) // Alg. 3
6   // Evaluate visibility for initial candidates
7   foreach pixel  $q \in$  Image do
8     | if shadowed(reservoirs[ $q$ ]. $y$ ) then
9       | | reservoirs[ $q$ ]. $W \leftarrow 0$ 
10  // Temporal reuse
11  foreach pixel  $q \in$  Image do
12    |  $q' \leftarrow$  pickTemporalNeighbor( $q$ )
13    | reservoirs[ $q$ ]  $\leftarrow$  combineReservoirs( $q$ , reservoirs[ $q$ ],
14      | | prevFrameReservoirs[ $q'$ ]) // Alg. 4
15  // Spatial reuse
16  for iteration  $i \leftarrow 1$  to  $n$  do
17    | foreach pixel  $q \in$  Image do
18      | |  $Q \leftarrow$  pickSpatialNeighbors( $q$ )
19      | |  $\mathbb{R} \leftarrow \{\text{reservoirs}[q'] \mid q' \in Q\}$ 
20      | | reservoirs[ $q$ ]  $\leftarrow$  combineReservoirs( $q$ , reservoirs[ $q$ ],  $\mathbb{R}$ )
21  // Compute pixel color
22  foreach pixel  $q \in$  Image do
23    | Image[ $q$ ]  $\leftarrow$  shadePixel(reservoirs[ $q$ ],  $q$ ) // Alg. 3
24  return reservoirs

```

图 8: RIS With Spatiotemporal Reuse

4.3.3 时空复用中的偏差

时空复用让当前 pixel 从其它 pixel 处获取样本，但是不同 pixel 对应的目标分布可能不一样，RIS 从不同的 PDF 处获取样本，这不可避免地会引入偏差。应对方法如图9 所示，证明方法见原论文。在进行时空复用后，对于蓄水池输出的样本，观察该样本在所用复用像素的样本空间中是否大于 0 (包含 visibility 测试)，保证时空复用的无偏，但是同时也会带来更多的性能消耗。

Algorithm 6: Unbiased combination of multiple reservoirs.

Input : Reservoirs r_i and the pixels q_i they originated from.
Output: An unbiased combination of the input reservoirs.

```

1 function combineReservoirsUnbiased( $q, r_1, r_2, \dots, r_k, q_1, \dots, q_k$ )
2   Reservoir  $s$ 
3   foreach  $r \in \{r_1, \dots, r_k\}$  do
4      $s.\text{update}(r.y, \hat{p}_q(r.y) \cdot r.W \cdot r.M)$ 
5    $s.M \leftarrow r_1.M + r_2.M + \dots + r_k.M$ 
6    $Z \leftarrow 0$ 
7   foreach  $q_i \in \{q_1, \dots, q_k\}$  do
8     if  $\hat{p}_{q_i}(s.y) > 0$  then
9        $Z \leftarrow Z + r_i.M$ 
10   $m \leftarrow 1/Z$ 
11   $s.W = \frac{1}{\hat{p}_q(s.y)} (m \cdot s.w_{\text{sum}})$  // Equation (20)
12  return  $s$ 

```

图 9: Unbiased Combine Multiple Reservoirs

4.4 参考

Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Resampled Importance Sampling

5 Rendering Many Lights with Grid-based Reservoirs

5.1 基本思想

本算法出自 Ray Tracing Gems2 中的第 23 章，是在 ReSTIR 算法基础上进行扩展。ReSTIR 应用于屏幕空间，主要在于 primary ray 的灯光采样；本论文更进一步，在世界空间内创建体素网格（grid），在每个 grid cell 内创建蓄水池。主要分为两步，第一步根据 grid cell 的位置信息初始化每个 grid cell 内部的光源样本；第二步，在着色点处根据 BRDF 的贡献进行重采样。

5.2 算法流程

5.2.1 初始化光源样本

对于每个体素，创建一定数量的光源样本，采样概率根据所有光源的 intensity（考虑光源到 grid cell 位置距离的衰减，距离裁剪到 grid cell 的边界，否则 grid cell 内的光源有相同的采样概率）。

每个 grid cell 内部有多个蓄水池，论文中称作 light slot，每个 light slot 包含一个光源样本。论文中默认实现为创建有 $16 \times 16 \times 16 = 4096$ 个 cell 的 grid，每个 cell 内部有 512（根据场景光源数量选择，64-1024）个 light slot，如图10所示。

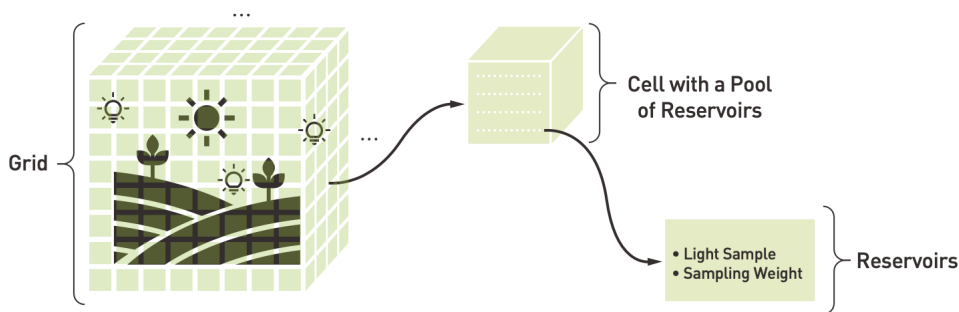


图 10: Restir Grid

(1) 摆放 grid

对于如何在场景中摆放 grid，论文给出了两种策略：

- 以相机位置为中心创建 clipmap，确定 cell 的尺寸，尽量避免出现漏光问题
- 使用 hash map 创建稀疏 grid

(2) 构建 light slot

对于每个 light slot，将 WRS 和 RIS 相结合生成光源样本，论文中设置 RIS 的 M（候选样本数量）为 8，此处每个 light slot 可以并行处理，充分利用 GPU 的并行处理能力。

每个 light slot 的采样策略如下：

```
1 float3 lightVector = candidate.position - gridCellCenter;  
2 float lightDistanceSquared = max(gMinDistanceSquared, dot(lightVector,  
3 lightVector));  
4 float sourcePdf = <Source PDF as described, e.g., uniform sampling>;  
5 float targetPdf = sample.intensity / lightDistanceSquared;
```

先通过均匀随机采样获取候选的光源样本，然后根据光源强度以及距离衰减对候选样本进行采样，如图11所示。

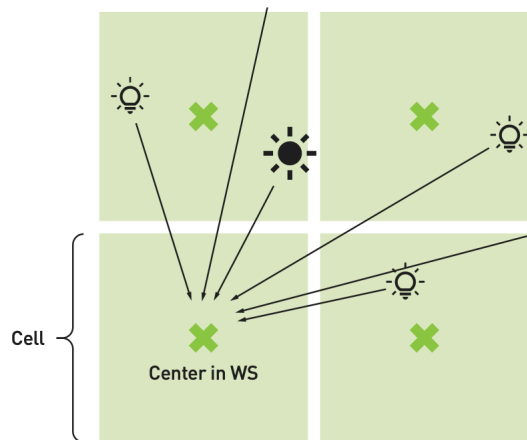


图 11: Light Slot Sample

(3) 时空复用

对历史帧创建的 light slot 进行复用，论文中设置为前 8 帧，将历史帧的 light slot 和当前帧的 light slot 进行合并，并始终保持 light slot 的 M 为 1:

```

1 // Merge new reservoir with reservoirs from previous frames.
2 for (int i = 0; i < GRIDS_HISTORY_LENGTH ; i++) {
3     lightSlotReservoir = mergeReservoirs(lightSlotReservoir ,
4     gLightGridHistory[i][lightSlotIndex]);
5
6     // Divide wSum by M after each combining "round ."
7     lightSlotReservoir.totalWeight /= lightSlotReservoir.M;
8     lightSlotReservoir.M = 1;
9 }

```

(4) 动态光源

当进行时序复用时，如果有光源发生变化，无法改变历史样本的权重，会出现噪点。对于这种情况，有两种策略：

- 抛弃动态光源的历史样本
- 重新计算动态光源的历史样本的权重

5.2.2 对 grid 进行采样

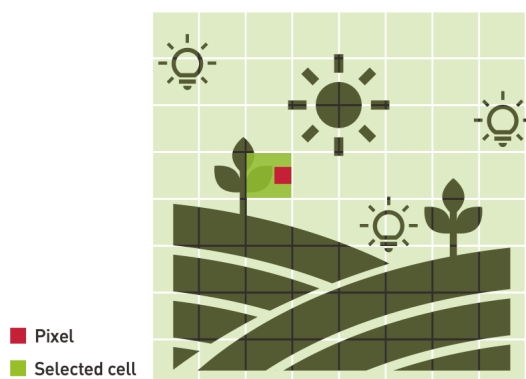


图 12: Sample the Grid

如图12所示, 根据着色点的位置查找 grid cell。我们先对着色点进行一个 jitter (相当于一个随机的三线性过滤), 减少离散数据存储带来的 artifact。然后根据 grid 信息获取当前 grid cell 对应的 light slot 索引, 每个 light slot 相当于一个蓄水池, 应用 RIS 对蓄水池进行合并, 得到最后的光源样本。

根据目标 PDF (着色点的 BRDF) 进行重采样:

```
1 float sourcePdf = candidate.sampleTargetPdf / reservoirAverageWeight;  
2 float targetPdf = PartialBrdf(candidate, shadedPoint);
```

最终的着色计算如下:

```
1 Reservoir risReservoir = sampleLightUsingRIS(shadedPoint);  
2 float lightSampleWeight = (risReservoir.totalWeight / M) / risReservoir.  
3 sampleTargetPdf ;  
4 LightSample light = loadLight(risReservoir.lightSample);  
5 return light.intensity * Brdf(light, shadedPoint) * lightSampleWeight;
```

5.3 参考

Ray Tracing Gems2